模块与包

模块化编程

在计算机程序的开发过程中，随着程序代码越写越多，在一个文件里代码就会越来越长，越来越不容易维护。

为了编写可维护的代码，我们把很多函数分组，分别放到不同的文件里，这样，每个文件包含的代码就相对较少，很多编程语言都采用这种组织代码的方式。

通过模块化编程，我们能把大的工程拆分成小的子任务和子模块，在比较大的项目中，进行模块化编程的好处有以下几点：

1. 简化编程，不必把重点放在整个项目上了，可以专注于正在处理的子任务。
2. 可维护性好，把问题限制在子任务中，即使出了问题也便于排查。
3. 复用性好，直接使用编写好的模块去实现功能，当需要重复实现时，再次调用即可，不必再重新编写。我们在编写程序的时候，也经常引用其他模块，包括Python内置的模块和来自第三方的模块。
4. 范围性好。使用模块还可以避免函数名和变量名冲突。相同名字的函数和变量完全可以分别存在不同的模块中，就是每个模块都有单独的命名空间，避免发生一些例如变量命名上的冲突。

我们自己在编写模块时，不必考虑名字会与其他模块冲突。但是也要注意，尽量不要与内置函数名字冲突。在编程中多使用函数，模块和包，能够提升模块化编程。

命名

每个变量都拥有一个名字，这个就是命名，给变量命名。变量命名也是让很多程序员头疼的一件事情，怎么样能起一些有意义，又高大上的名字。在Python中，一切皆对象，我们甚至可以给一个函数一个命名，命名就可以理解为所有对象的一个引用的名称。

命名空间

命名空间就是用来保证命名之间不发生冲突的规则，分为：

- 局部命名空间：在一个程序中为每个函数创建的命名空间，当函数执行完毕就失效
- 全局命名空间：在程序中为所有模块创建的命名空间，当程序执行完毕失效
- 内置命名空间：也就是默认的一些命名，退出程序失效

模块(module)

模块，就是一个扩展名为“.py”的python源代码文件。模块分为内置模块、第三方模块和自定义模块。

- 内置模块，直接导入就可以了。例如：math, random等 。
- 第三方模块，需要安装后再导入。例如：numpy, matplotlib等。
- 自定义模块，需要自己写一个python文件。

模块一般是用 Python 语言编写的，也有些模块是用 C 语言实现，在运行时动态加载，比如常用的正则表达式模块 `re`；

导入模块

Python 使用 import 导入模块。常用的写法：

将某个模块整个导入

语法：import 模块 。导入后需要使用"模块名.函数名()"调用函数。

```
# 1. 导入模块
# import 模块名 （建议一行只导入一个模块）
# import 模块名1, 模块名2... （不建议一行导入多个模块）

# 2. 调用功能
# 模块名.功能名()

import math
print(math.sqrt(9)) # 3.0
```

将某个模块的某个函数导入

语法：from 模块 import 函数。导入后直接使用"函数名()"调用函数。

```
# 导入功能函数
# from 模块名 import 功能1, 功能2, 功能3...（导入模块中部分函数）
# from 模块名 import * （导入模块中所有的函数）

from math import sqrt
print(sqrt(9))

from math import *
print(sqrt(9))
```

as定义别名

- 将某个模块整个导入时加别名。语法：`import 模块 as 别名`。导入后需要使用"模块别名.函数名()"调用函数。
- 将某个模块的某个函数导入时加别名。语法：`from 模块 import 函数 as 别名`。导入后直接使用"函数别名()"调用函数。

```
# 模块定义别名
# import 模块名 as 别名
# import 模块名1 as 别名1, 模块名2 as 别名2 ……

# 函数定义别名
# from 模块名 import 函数 as 别名
# from 模块名 import 函数1 as 别名1, 函数2 as 别名2 ……

# 模块别名
import time as tt

tt.sleep(2)
```

```
print('hello')

# 函数别名
from time import sleep as sl
sl(2)
print('hello')
```

导入建议

- 不要使用 from ... import *

  有人觉得使用 from ... import * ，直接把一个模块中所有的函数都导入到当前命名空间中，函数可以直接使用而不需要加模块名，很省事。但是，这可能会导致函数名冲突，这被称做命名空间污染，是要尽量避免的。原因如下：

  如果使用 from ... import ... 或 from ... import * 导入多个模块的时候，且模块内有同名功能。当调用这个同名功能的时候，调用到的是后导入的模块的功能。

  举例：假如你的程序是这样的：

```
from some_module import *
print("Hello World!")
```

  结果程序没有任何输出.....

  打开 some_module.py 一看，程序是这样的：

```
# some_module.py
def print(*args):
    exit(1)
```

- import 时一行只导入一个模块，不要一行导入多个模块.

创建模块

1. 自己写一个python文件;
2. 用C语言实现，然后在运行时动态加载，比如常用的正则表达式模块 `re`;

这里，我们主要关注第一种方式，也就是说，`python`给我们提供了十分简单的方法去创建一个模块，我们只需要写一个`python`文件即可，也就是说写一个`.py`为后缀的文件，不必用额外的语法。

举个例子，我们现在写了一个文件，命名为`mod.py`，如下：*mod.py*

```
s = "If Comrade Napoleon says it, it must be right."
a = [100, 200, 300]

def foo(arg):
    print(f'arg = {arg}')

class Foo:
    pass
```

可以看到，我们在这个python文件里面创建了一下几个对象：

- s，一个字符串
- a，一个列表
- foo，一个函数
- Foo，一个类

假定这个文件路径正确，我们就可以把`mod.py`作为一个模块，在别的文件中引用，举例：

```
>>> import mod
>>> print(mod.s)
If Comrade Napoleon says it, it must be right.
>>> mod.a
[100, 200, 300]
>>> mod.foo(['quux', 'corge', 'grault'])
arg = ['quux', 'corge', 'grault']
>>> x = mod.Foo()
>>> x
<mod.Foo object at 0x03C181F0>
```

模块搜索路径

继续上面的例子，当执行下面这句的时候，解释器都干了什么？

```
import mod
```

解释器主要是在如下几个路径中搜索`mod.py`文件：

1. 首先在当前执行的文件所在的文件夹中查看是否存在，如果当前文件夹不包含`mod.py`文件，那么这个搜索就失败了；
2. 在系统设置的python环境变量下的一些文件夹里面；
3. 一些在python安装时指定的文件夹下面；

具体我们可以使用：

```
>>> import sys
>>> sys.path
['', 'C:\\Users\\john\\Documents\\Python\\doc', 'C:\\Python36\\Lib\\idlelib',
'C:\\Python36\\python36.zip', 'C:\\Python36\\DLLs', 'C:\\Python36\\lib',
'C:\\Python36', 'C:\\Python36\\lib\\site-packages']
```

来查看在路径搜索时，都有哪些路径，当然搜索的结果因为安装，版本和操作系统的原因都不会相同，每个人的结果可能都不会相同。

因此，当我们`import`一些模块出现问题的时候，我们可以查看这个模块文件是不是在搜索路径中。

当我们想要把自己写的文件作为一个模块的时候，但是又不在搜索路径中，我们可以自己添加进去，比如：

```
>>> sys.path.append(r'C:\Users\john')
>>> sys.path
['', 'C:\\Users\\john\\Documents\\Python\\doc', 'C:\\Python36\\Lib\\idlelib',
'C:\\Python36\\python36.zip', 'C:\\Python36\\DLLs', 'C:\\Python36\\lib',
'C:\\Python36', 'C:\\Python36\\lib\\site-packages', 'C:\\Users\\john']
>>> import mod
```

因为`sys.path`返回的是路径的列表，我们直接使用`append()`方法把想要引用的模块路径添加进去即可。

当作脚本执行

如果想让python文件即可以被当作模块引用，也能被当作脚本执行，比如前面的`mod.py`文件，

```
s = "If Comrade Napoleon says it, it must be right."
a = [100, 200, 300]

def foo(arg):
    print(f'arg = {arg}')

class Foo:
    pass
```

当我们单独执行者文件的时候，是没有结果的输出的。

```
s = "If Comrade Napoleon says it, it must be right."
a = [100, 200, 300]

def foo(arg):
    print(f'arg = {arg}')

class Foo:
    pass

print(s)
print(a)
foo('quux')
x = Foo()
print(x)
```

当执行这个文件时，有结果输出。但是当我们把这个文件当作一个模块，在其他文件中引用时，同样的结果会输出出来，这让不是我们需要的结果。

正确的做法是：

```
s = "If Comrade Napoleon says it, it must be right."
a = [100, 200, 300]

def foo(arg):
    print(f'arg = {arg}')

class Foo:
```

```
    pass

if __name__ == '__main__':
    print('Executing as standalone script')
    print(s)
    print(a)
    foo('quux')
    x = Foo()
    print(x)
```

加上了`if __name__ == '__main__':`，这样python解释器就知道这个文件时单独在执行，还是被作为模块引用。在单独执行时，有输出，而在作为模块引用时，没有输出。

例子：日期工具模块

```
# MyDateUtils.py

def isLeapYear(year):
    return year%4==0 and year%100 != 0 or year%400 == 0

def daysOfYear(year):
    return 366 if isLeapYear(year) else 365

def daysOfMonth(year,month):
    DAYS_OF_MONTHS = (31,28,31,30,31,30,31,31,30,31,30,31)
    if month==2:
        if isLeapYear(year):
            return DAYS_OF_MONTHS[month-1]+1
        else:
            return DAYS_OF_MONTHS[month-1]
    else:
        return DAYS_OF_MONTHS[month-1]

def date_to_days(year,month,day):
    pass

def days_to_date(days):
    pass
```

包(package)

模块可以避免变量、函数和类名重名，但是模块名重复怎么办？例如，不同的人编写的模块名相同。为了避免模块名冲突，Python又引入了按目录来组织模块的方法，称为包（Package）。包可以避免模块重名。

简单来说，包就是多个模块的集合。当项目较大，模块较多时，我们就可以把模块放在包中，便于管理。包就是一个目录（文件夹），使用“包名.模块名”就可以访问到模块了，即在引用包中的模块时，使用`.`操作符。

使用包之前，与模块一样需要导入，同样也要注意是不是在搜索路径中。

包的初始化工作

包的定义很简单，在当面目录下有 `__init__.py` 文件的目录即为一个包。分为两种情况，第一种情况是一个空的 `__init__.py` 文件，另外一个情况是写了代码的 `__init__.py` 文件。 不管是空的还是有内容的，这个目录都会被认为是一个包,这是一个标识。

当初始化一个包需要配置时，我们在 `__init__.py` 文件中写入一些指令，如果不需要的话，空文件也可以。一个包被导入，不管在什么时候 `__init__.py` 的代码都只会被执行一次。

由于包被导入时 `__init__.py` 中的可执行代码会被执行，所以小心在包中放置你的代码，尽可能消除它们产生的副作用，比如把代码尽可能的进行封装成函数或类。

导入顺序

当尝试导入一个包时

```
from package import something
```

`import` 语句会首先检查 `something` 是不是 `__init__.py` 的变量，然后检查是不是 `subpackage`，再检查是不是 `module`，最后抛出 `ImportError` 。所以检查顺序如下：

1. `__init__.py` 文件内变量
2. 是不是package内的 `subpackage`
3. 是不是package内的 `module`

一个简单的例子

我们有一个如下结构的包：

```
pkg
|-- a.py
|-- b.py
`-- __init__.py
```

在 `a.py` 文件内有一个函数

```
def bar():
    print("Hello, function 'bar' from module 'a' calling")
```

在 `b.py` 文件内有一个函数

```
def foo():
    print("Hello, function 'foo' from module 'b' calling")
```

放一个空的 `__init__.py` 文件在 `pkg` 里面。

当我们 `import pkg` 的时候到底会发生什么事情(在 `pkg` 内激活 `Python shell` 或者 `pkg` 的路径被包含在 `python` 的 `sys.path` 或者在 `PYTHONPATH` 的环境变量中)

```
>>> import pkg
>>> pkg
<module 'pkg' from '/home/user/pkg/__init__.py'>
>>> pkg.a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: module 'pkg' has no attribute 'a'
```

我们可以看到`simple_package`已经被成功导入，但是`a.py`和`b.py`并没有被导入。如果希望使用`import simple_package`后自动加载`a`或者`b`模块，这里有两种办法。

- 第一种就是在`__init__.py`内导入`a`或者`b`模块，然后保存再激活`python`的交互环境

```
#__init__.py
import pkg.a
import pkg.b
```

当你再次尝试`import pkg`后，就可以使用`pkg.a.bar()`来使用模块`a`中的`bar()`函数了。

- 第二办法就是手动导入，当你想使用模块`a`中的`bar()`函数时，需要手动导入

```
import pkg.a as a
```

然后就是可以`a.bar()`来使用`bar()`函数了。

一个更复杂的例子

这是一个来自官方的例子。文件结构如下

```
sound
|-- effects
|   |-- echo.py
|   |-- __init__.py
|   |-- reverse.py
|   `-- surround.py
|-- filters
|   |-- equalizer.py
|   |-- __init__.py
|   |-- karaoke.py
|   `-- vocoder.py
|-- formats
|   |-- aiffread.py
|   |-- aiffwrite.py
|   |-- auread.py
|   |-- auwrite.py
|   |-- __init__.py
|   |-- wavread.py
|   `-- wavwrite.py
`-- __init__.py
```

你可以将这个package的例子下载下来。如果直接使用`import sound`来导入这个package，我们可以导入package `sound`，但是`sound`的子package（`effects`,`filters`,`formats`）并不会被自动导入。子package不会被自动导入的原因是因为在`sound`目录下的`__init__.py`文件并没有任何关于导入子package的代码。

我们来看下在`sound`目录下的`__init__.py`的代码

```
"""An empty sound package

This is the sound package, providing hardly anything!"""


print("sound package is getting imported!")
```

然后我们导入`sound`试下

```
>>> import sound
sound package is getting imported!
>>> sound
<module 'sound' from '/home/user/packages/sound/__init__.py'>
>>> sound.effects
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: module 'sound' has no attribute 'effects'
```

如果你想使用子package的内容，但是在父package的`__init__.py`的文件内并没有导入，你需要手动导入

```
>>> import sound.effects
effects package is getting imported!
>>> sound.effects
<module 'sound.effects' from '/home/user/packages/sound/effects/__init__.py'>
```

如果你希望`python`帮你自动导入`sound.effects`你可以往`sound`目录下的`__init__.py`文件写入

```
"""An empty sound package

This is the sound package, providing hardly anything!"""

import sound.effects
print("sound package is getting imported!")
```

那么你下次运行的时候`python`就会自动帮你导入`sound.effects`

```
>>> import sound
effects package is getting imported!
sound package is getting imported!
```

当然了，除了使用绝对路径你可以使用相对路径来导入`sound.effects`

```
"""An empty sound package

This is the sound package, providing hardly anything!"""

from . import effects
print("sound package is getting imported!")
```

这跟`linux`的命令行比较像，`.`代表当前目录，`..`代表上级目录。所以你可以在`sound.effects`的`__init__.py`文件内写入

```
from .. import formats
```

来导入`sound.formats`。

当你使用`sound`的时候就会发现，`sound.effects`和`sound.formats`都被导入了。

```
>>> import sound
formats package is getting imported!
effects package is getting imported!
sound package is getting imported!
```

最后展示一下，怎么从`sound.effects`导入`sound.filters.karaoke`模块，将一下代码加入到`sound.effects`的`__init__.py`文件中

```
"""An empty effects package

This is the effects package, providing hardly anything!"""

from .. import formats
from ..filters import karaoke
print("effects package is getting imported!")
```

激活`python`的交互环境以后，尝试`import sound`

```
>>> import sound
formats package is getting imported!
filters package is getting imported!
Module karaoke.py has been loaded!
effects package is getting imported!
sound package is getting imported!
```

现在我们可以使用`karaoke`的函数了

```
>>> sound.filters.karaoke.func1()
Funktion func1 has been called!
>>>
```

把整个包都导入进来

还是用前面的例子，我们尝试使用`*`来进行全部的导入sound下的所有模块

```
>>> from sound import *
sound package is getting imported!
```

我们可以看到仅仅是导入了`sound`这个package但是其他的内容并没有导入。使用dir()函数查看可知。

> dir() 函数一个排好序的字符串列表，内容是一个模块里定义过的名字。返回的列表容纳了在一个模块里定义的所有模块，变量和函数。

```
>>> dir()
['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
```

`__all__`

我们可以使用`__all__`这个魔法变量来手动导入模块和子package，当你定义了`__all__`到`__init__.py`文件以后，`python`会根据你在list内给出的元素进行逐个导入

```
__all__ = ["formats", "filters", "effects"]
```

所以我们可以再次导入试试

```
>>> from sound import *
sound package is getting imported!
formats package is getting imported!
filters package is getting imported!
effects package is getting imported!
```

看下`dir()`

```
>>> dir()
['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'effects',
'filters', 'formats']
>>>
```

你会发现所有模块都已经被顺利导入。

那如果我们仅仅导入`sound.effects`package内所有内容呢，会发生什么，我们`import`的时候到底`import`的是什么。

我们看下结果

```
>>> from sound.effects import *
sound package is getting imported!
effects package is getting imported!
>>> dir()
['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
>>>
```

你会发现他仅仅是导入了`sound.effects`这个package，跟你没有修改`sound`的`__init__.py`之前是类似情况，仅仅是导入了这个package。

所以可以修改`sound.effects`的`__init__.py`文件来导入`effects`内的所有模块。

```
__all__ = ["echo", "surround", "reverse"]
```

看下结果

```
>>> from sound.effects import *
sound package is getting imported!
effects package is getting imported!
Module echo.py has been loaded!
Module surround.py has been loaded!
Module reverse.py has been loaded!
>>>
>>> dir()
['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'echo',
'reverse', 'surround']
>>>
```

小结

1. 在`from package import *`语句中，如果`__init__.py`中定义了`__all__`魔法变量，那么在`__all__`内的所有元素都会被作为模块自动被导入（如果自动导入的模块不存在的话，`ImportError`仍然会出现）。
2. 如果

   ```
   __init__.py
   ```

   中没有

   ```
   __all__
   ```

   变量，导出将按照以下规则执行：

   1. 此 package 被导入，并且执行`__init__.py`中可被执行的代码
   2. `__init__.py`中定义的 variable 被导入
   3. `__init__.py`中被显式导入的 module 被导入

安装第三方的模块和包

Python中的模块和包，分为内置的标准库和第三方的库。内置的标准库直接导入即可使用，常见的有 os, sys, math, random, re, time, datetime, copy, csv, json, sqlite3, urllib 等。以下是一些常用标准库的说明。

标准库	说明
os 模块	os 模块提供了许多与操作系统交互的函数，例如创建、移动和删除文件和目录，以及访问环境变量等。

标准库	说明
sys 模块	sys 模块提供了与 Python 解释器和系统相关的功能，例如解释器的版本和路径，以及与 stdin、stdout 和 stderr 相关的信息。
time 模块	time 模块提供了处理时间的函数，例如获取当前时间、格式化日期和时间、计时等。
datetime 模块	datetime 模块提供了更高级的日期和时间处理函数，例如处理时区、计算时间差、计算日期差等。
random 模块	random 模块提供了生成随机数的函数，例如生成随机整数、浮点数、序列等。
math 模块	math 模块提供了数学函数，例如三角函数、对数函数、指数函数、常数等。
re 模块	re 模块提供了正则表达式处理函数，可以用于文本搜索、替换、分割等。
json 模块	json 模块提供了 JSON 编码和解码函数，可以将 Python 对象转换为 JSON 格式，并从 JSON 格式中解析出 Python 对象。
urllib 模块	urllib 模块提供了访问网页和处理 URL 的功能，包括下载文件、发送 POST 请求、处理 cookies 等。

第三方的库则需要额外安装后方可使用。

可以使用 `help()` -> `help> modules` 查看已安装的模块与包。

```
>>> help()
help> modules
```

常见的包可以查看PyPI（Python Package Index）：https://pypi.org/，通过使用 pip install 来安装社区提供的Packages。

- 第三方模块 通常是指由 知名的第三方团队开发的并且被程序员广泛使用 的 Python 包 / 模块。例如 NumPy 是使用 Python 进行科学计算的基本软件包。
- pip 是一个现代的，通用的 Python 包管理工具，提供了对 Python 包的查找、下载、安装、卸载等功能。

```
pip install numpy
```

如果想打包自己的Package并分发，可以通过 setuptools 工具。使用 pyinstaller 可以生成一个安装包便于用户安装。

总结

我们主要讨论了如何创建模块，使用模块的一些细节以及包的相关概念，当我们创建了模块，我们就可以在别的程序中复用自己模块，当模块很多时，我们就可以把模块放入一个包中进行管理。